STORAGE ENGINE PERFORMANCE · ROCKSDB 11.2.0

RocksDB 写性能分析

默认配置下 1KB KV / 100GB 写入负载的 CPU 归因、内存构成与瓶颈解剖：写路径 74% 的 CPU 中有效做功仅约 5%，其余为并发协调与限速节流下的自旋等待；后台 compaction 线程 1→6 使 fill 墙钟缩短 24%，但 16MB/s 的默认限速上限仍是主约束。

实验环境：i9-10980XE · 36 线程 · 62 GB 内存 · NVMe SSD | 报告日期：2026 年 10 月

目录

目录 2
1 摘要与关键结论 3
五条可执行的结论： . . . 3
2 实验设置与测量口径 4
2.1 负载与配置 . . . 4
2.2 四个语料的对照关系 . . . 4
2.3 可信度对账 . . . 4
3 写吞吐：墙钟、时间线与节流形态 5
3.1 限速把时间与数据“错配” . . . 5
4 瓶颈解剖：compaction 积压 → 限速 → 前台等待 6
5 CPU 归因：谁在消耗算力 7
6 写路径的并发成本：自旋与协调 8
7 后台线程数的杠杆与代价 9
8 内存构成 10
9 线程状态与函数级剖面 10
10 结论与优化建议 11
10.1 本报告的局限 . . . 12
附录 A 详细数据表 13
A.1 三个 100GB 跑的总览 . . . 13
A.2 CPU 归因明细（核心·秒） . . . 13
A.3 写线程扫描（wpbench，quiet，2M ops） . . . 13
A.4 写路径组件与开关（wpbench） . . . 13
A.5 后台线程 / 子压缩扫描 . . . 14
A.6 内存构成（MB） . . . 14
A.7 stall 事件与原因 . . . 14
A.8 perf 自时间热点（%） . . . 15
A.9 生效的默认配置（库自报） . . . 15

1 摘要与关键结论

本报告基于同一环境的四组实测（三个 100GB 跑 + 一轮参数扫描矩阵），回答默认配置下 1KB KV / 100GB 写入的 CPU 去向、内存构成与吞吐瓶颈。所有数据来自自研驱动 bench/fillcpu（只设 create_if_missing，不开 Statistics）与 bench/wpbench（逐 op CPU 与 PerfContext 拆解），并用库侧 rocksdb.cfstats 与逐线程 /proc 快照双向对账。

一句话结论：写入本身很便宜（5.8 µs/KV，约占全程 CPU 的 5%），真正昂贵的是“被节流时如何等待”——默认 16 MB/s 的限速上限与 64 GB 的 compaction 积压软限把 3/4 的墙钟钉在涓流状态，而 16 个写线程在此期间持续自旋，把等待算成了 CPU。

22.9→30.3 MB/s	−24%	5.8 µs/KV	1.8–2.1%	979→1405 MB
fill 吞吐（bg2→bg8）	fill 墙钟（4156→3148 s）	单线程有效写功	flush 占全程 CPU	峰值 RSS（bg2→bg8）

五条可执行的结论：

- 写路径 CPU 的九成是并发协调与自旋，不是写数据。有效写功（WAL + memtable + 驱动）实测 5.76 µs/KV；16 线程下写路径 CPU 升到 31.1 µs/op，75% 的机队时间在 WriteThread::AwaitState 里自旋（perf 剖面：sched_yield + 内核系统调用帧 ~20%，memtable 插入仅 ~2.5%）。
- 真正的吞吐瓶颈是 compaction 积压与限速上限。100GB 跑中 73.8–74.9% 的墙钟速率 ≤16.5 MB/s（中位 11 MB/s），因为 pending compaction bytes 长期压在 64 GB 软限之上；约 26% 的时间放开全速时能跑到 62–97 MB/s（峰值 623 MB/s），贡献 70–85% 的数据。
- 加 compaction 线程有用但会饱和：1→6 线程把 fill 缩短 24%（4156→3148 s）、WAF 从 9.55× 降到 8.33×、积压限速事件 −45%；但 16MB/s 限速上限仍在 74% 的时间里生效，且 6 路并发把磁盘占满（compaction 线程 65% 时间处于 D 状态），代价是 flush 单作业延迟 ×2.8、内存 +39%。
- 不要把 max_subcompactions 当作提速手段：实测写放大 +35%、吞吐 −25~34%（8GB 规模），线程数膨胀并大量陷入 IO 等待。
- 写线程数不是越多越好：吞吐在 16 线程时 3.3× 封顶（单线程 11551 ops/核·秒 → 16 线程 7947，效率 −31%），32 线程因超出物理核数而塌陷到单线程水平以下；后台受限时应减线程——8 线程比 16 线程省 34% CPU、只慢 10%。

2 实验设置与测量口径

2.1 负载与配置

硬件：Intel i9-10980XE（18 核 / 36 线程）、62 GB 内存、NVMe SSD（O_DIRECT 顺序写实测 ~950 MB/s）。软件：RocksDB 11.2.0（未修改源码，链接 librocksdb.so.11.2.0），本机构建未链接压缩库，因此默认压缩解析为不压缩。负载为 96,000,000 键 × (16B 键 + 1024B 值) = 99.84 GB 应用数据，16 个写线程按全键空间伪随机序写入（fill），随后停写并等待引擎静止（drain），以便把全部 flush/compaction 计完。

测量边界：驱动仅设置 create_if_missing 与一个只读 EventListener（用于逐作业计时），不开启 Statistics，因此被测的就是库默认配置；线程级 CPU 取 /proc/self/task/<tid>/stat 的 utime+stime 精确快照（含 R/S/D 状态），flush 逐作业 CPU 用 CLOCK_THREAD_CPUTIME_ID 在 OnFlushBegin→OnFlushCompleted 窗口内夹逼，compaction 同时记录库自报的 CompactionJobStats.cpu_micros。

2.2 四个语料的对照关系

语料	配置差异	外部负载	用途
bg2 · 归档	默认：1 flush + 1 compaction 线程	load 2–3	7 章 bg2 基线旧参考
bg2 · 匹配	同上（与 bg8 相邻时段重跑）	load ≈3	本报告主基线
bg8	max_background_jobs=8（2 flush + 6 compaction）	load 5–13	本轮重点 + perf 采样
扫描矩阵	wpbench 参数扫描（线程数 / WAL / 后台 / 流水线）	load 2–9	拆解成本用

表 2-1 本报告使用的四组数据。外部负载指机器上的其它作业（共享机器），bg8 的优势是在更高负载下取得的，故为保守下界。

2.3 可信度对账

对账项（100GB 归档跑）	口径 A	口径 B	差异
进程 CPU vs 线程和	11678.08 s	11677.85 s	0.003%
compaction：驱动夹逼 vs 库自报	2564.87 s	2550.34 s	0.6%
flush：驱动作业窗口 vs cfstats(High)	163.78 s	160.58 s	2.0%
flush 输出字节 vs 应用字节	100.23 GB	99.84 GB	0.4%

表 2-2 三路独立口径一致，说明逐线程 / 逐作业归因可信。

3 写吞吐：墙钟、时间线与节流形态

图 2　100GB fill 的墙钟与吞吐：bg8 比匹配基线快 24%

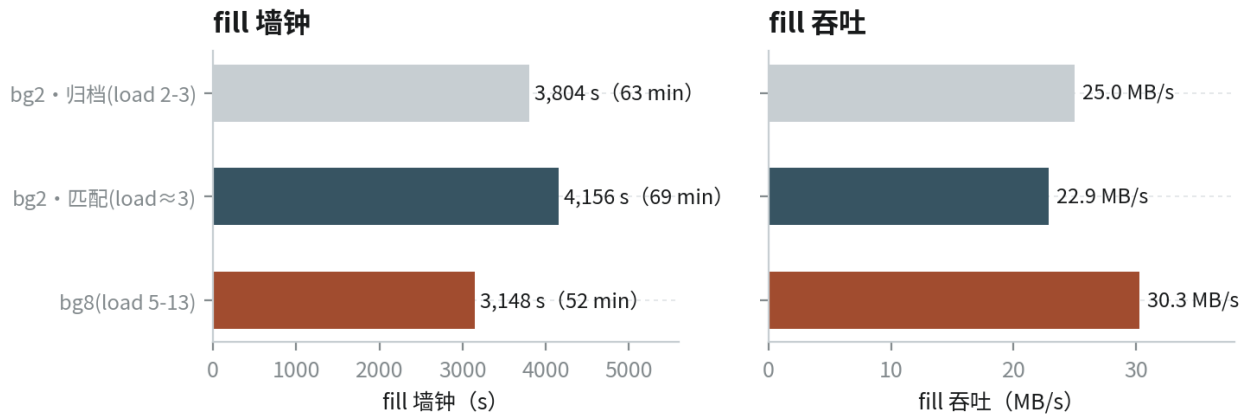


图 2 fill 墙钟与吞吐对比（96M×1KB）。bg8 用 6 个 compaction 线程把 fill 从 4156 s 缩到 3148 s。

图 1 是三个跑的 10 秒窗口写速率。可以看到两条特征：① 大部分时间速率贴着 16 MB/s的限速上限（红色虚线）以下涓流；② 每隔一段时间出现一次爆发（200–620 MB/s），把积压的 memtable 一次性灌满。这种“涓流—泄洪”交替就是默认配置下 100GB 写入的稳定形态。

图 1　100GB fill 的写速率时间线：限速涓流 + 周期性泄洪

96M×1KB；bg2=默认 1+1 后台线程，bg8=2 flush + 6 compaction 线程

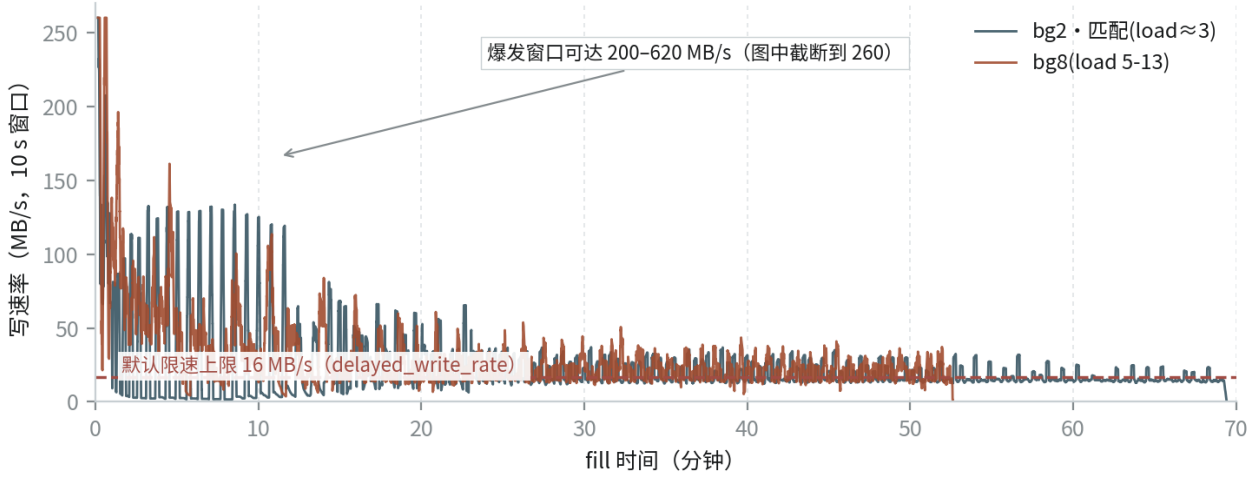


图 1 写速率时间线：95% 以上的窗口速率低于 100 MB/s，爆发窗口在图中截断到 260 MB/s。

3.1 限速把时间与数据“错配”

把 fill 的每个 250ms 区间按速率是否 ≤16.5 MB/s 分成两桶，结果是：约 3/4 的时间被限速，但只有 15–31% 的数据是在限速区间写掉的——换句话说，写够 100GB 靠的是少数全速窗口。这也解释了为什么加写线程对总吞吐几乎没有贡献：限速状态下速率由控制器决定，而非前台线程数。

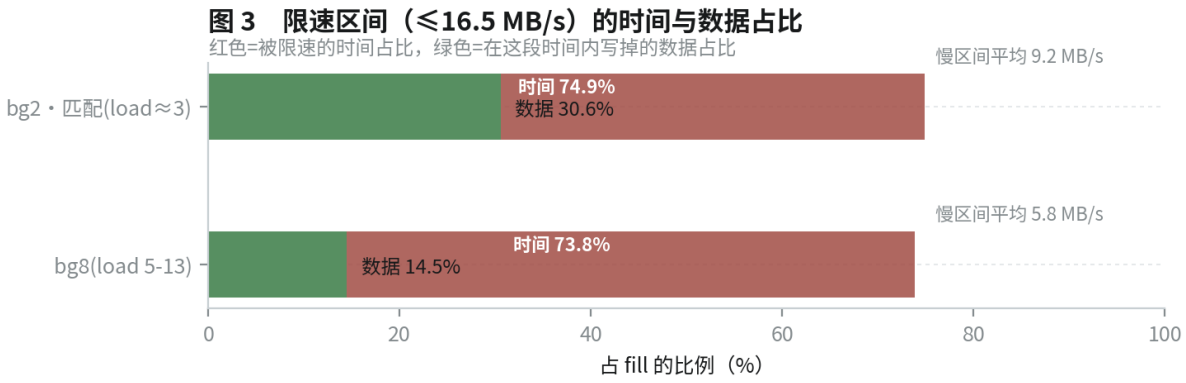


图 3 限速区间的时间占比 vs 数据占比。bg8 的慢区间更慢（自适应速率被压得更低）、快区间更快（6 线程消化积压更快）。

4 瓶颈解剖：compaction 积压 → 限速 → 前台等待

机制链条（可量化）

① flush 把数据送进 L0（100.2 GB / 1550–1563 个 SST）；

② 每 1 GB 进入 LSM 的数据在 100GB 规模要引发 8.3–9.5 GB 的 compaction 读写（WAF）；

③ 单线程 compaction 排空能力有限，待压缩字节累积并长期停留在 64 GB 软限之上；

④ 超过软限即安装限速令牌：写速率上限 = delayed_write_rate（默认 16 MB/s），积压增长时按 ×0.8 自适应下调（下限 16 KB/s）；

⑤ 前台按 1 ms 粒度睡眠等待，memtable 槽位耗尽时则完全停车（bg_cv_.Wait()）。

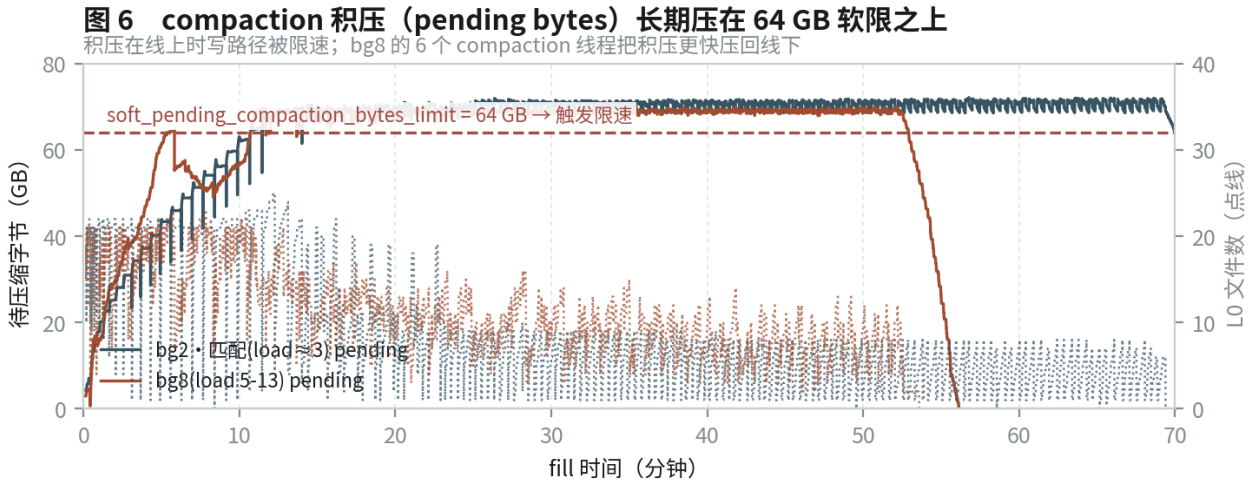


图 6 待压缩字节（实线）与 L0 文件数（点线）：积压长期贴着 64 GB 线上方运行，bg8 靠 6 个线程把波动压得更低。

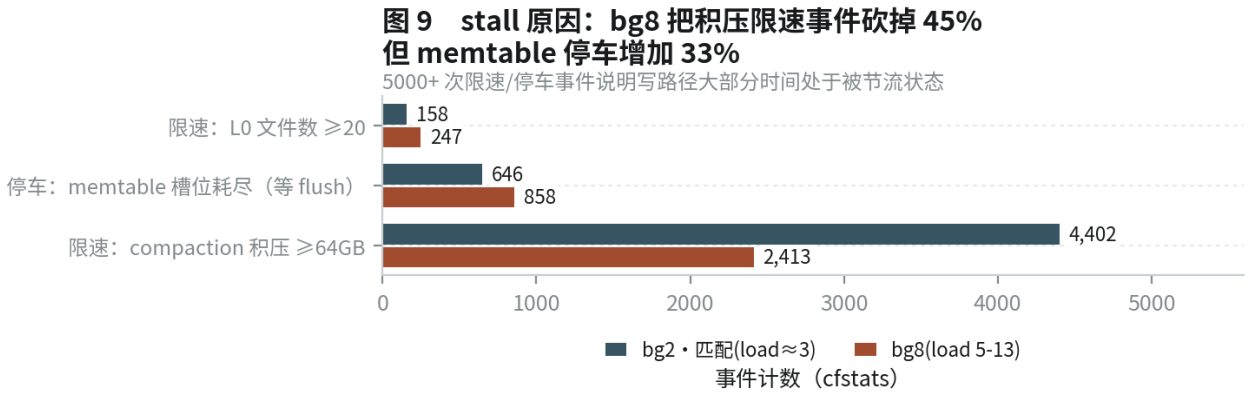


图 9 stall 事件计数：bg8 把“积压限速”砍掉 45%，但 memtable 停车反增 33%（磁盘被 compaction 占满，flush 变慢）。

需要强调术语：这里说的 stall 就是 RocksDB 的 write stall，由 WriteStallCondition（kDelayed / kStopped）与原因（memtable 上限 / L0 文件数 / 待压缩字节）组成。其中 kDelayed 是限速而非停写——100GB 跑里它占 96% 以上的 stall 时间，写仍在以 ~11 MB/s 进行；真正的停写只在 memtable 槽位耗尽时发生（646–858 次）。另有一类等待与 stall 无关：写组排队（自旋），任何并发下都存在。

5 CPU 归因：谁在消耗算力

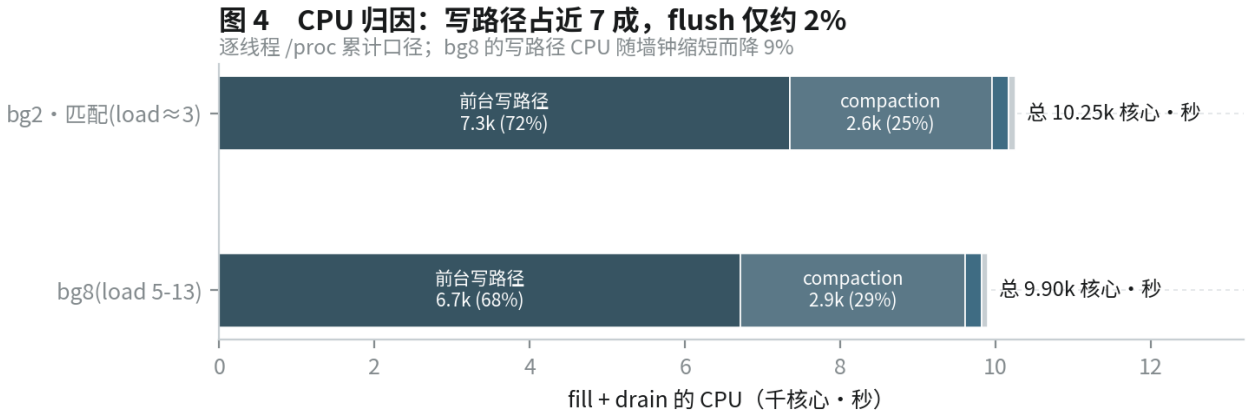


图 4 fill + drain 全程的 CPU 归因（逐线程 /proc 口径）。写路径约 72%，compaction 约 24%，flush 仅约 2%。

跑	总 CPU	fill/drain 墙钟	写路径	compaction	flush	写路径 µs/KV
bg2 · 匹配	10.25 k	4156 / 296 s	7.35 k（71.7%）	2.60 k（25.4%）	0.22 k（2.1%）	76.5 µs
bg8	9.90 k	3148 / 222 s	6.72 k（67.9%）	2.90 k（29.3%）	0.21 k（2.1%）	70.0 µs

表 5-1 CPU 归因（核心 · 秒）。bg8 的总 CPU 低 3.5%，但结构变化明显：写路径 −9%、compaction +9%。

flush 的账要分开算。100GB 数据对应的 flush 线程 CPU 只有 209–217 核心 · 秒（全程 2.1%），单作业中位 104–110 ms/64 MB；但它的间接影响大得多：flush 是 compaction 积压的唯一来源，而积压决定限速开关（第 4 章）。此外 flush 自身还触发 memtable 上限停车（646–858 次）。评估“减少 flush 工作量”的收益时，应把这两部分分开计价。

6 写路径的并发成本：自旋与协调

图 5　写线程数扫描：吞吐 3.3× 封顶，CPU/op 与延迟持续上升

1KB Put、无后台干扰（quiet）、2M ops；32 线程因超出物理核数而塌陷

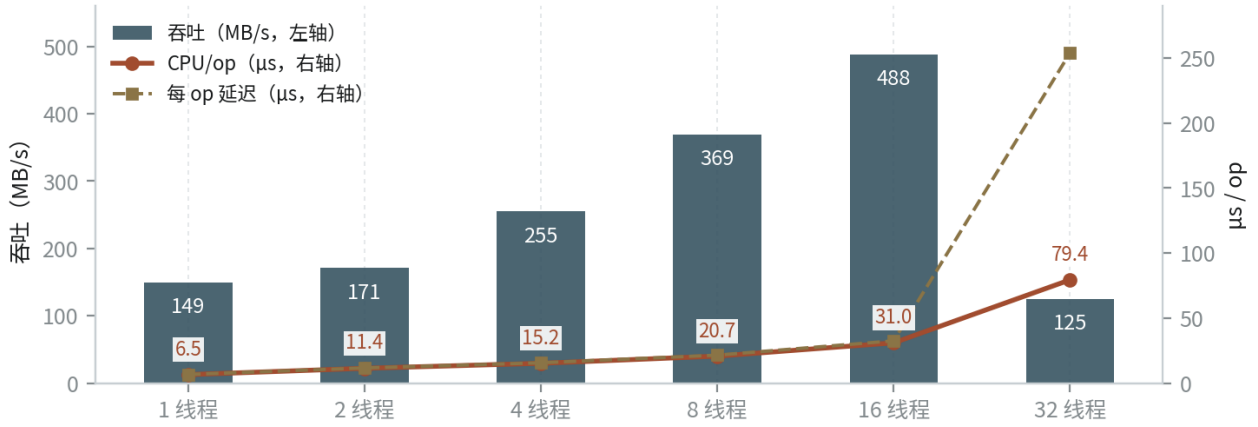


图 5 写线程数扫描（wpbench quiet，无后台干扰）：吞吐 1→16 线程 3.27×，CPU/op 4.8×，每 op 延迟 5×。

写线程	吞吐 MB/s	加速比	每 op 延迟	CPU/op	stime 占比	每核效率 ops/核·s
1	149	1.00×	6.6 µs	6.5 µs	17%	153,539
2	171	1.14×	11.6 µs	11.4 µs	24%	87,866
4	255	1.71×	15.6 µs	15.2 µs	32%	65,705
8	369	2.47×	21.5 µs	20.7 µs	42%	48,201
16	488	3.27×	32.5 µs	31.0 µs	50%	32,297
32	125	0.84×	254.0 µs	79.4 µs	36%	12,595

表 6-1 写线程扫描（1KB Put，quiet，2M ops）。效率最优是单线程，超过 16 线程后吞吐塌陷。

图 11　写路径开销拆解（wpbench，quiet）：有效功 5.76 µs（memtable 3.30 + WAL 1.10 + 驱动 1.36，浅色为 wal=0 对照 4.67）

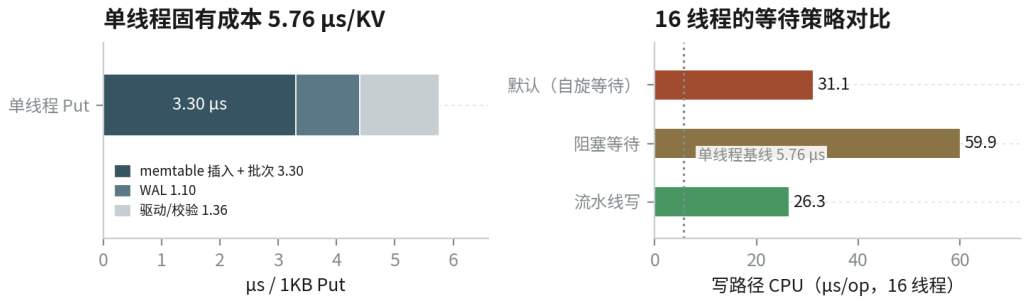


图 11 左：单线程写路径的成本拆解（5.76 µs/KV，其中 memtable 3.3、WAL 1.1）；右：16 线程下的三种等待策略对比。

自旋从哪来：WriteThread::AwaitState 的三段式等待（先 200 次 pause，再用 sched_yield 空转最多 100 µs，最后才阻塞），默认开启（enable_write_thread_adaptive_yield=true）。16 个写线程在 36 线程机器上 yield 总能立刻返回，于是持续自旋——逐线程 CPU 完全均匀、极差不到 0.4%（539.6–541.8 s）——这正是“对称自旋”的指纹。把等待从自旋改成阻塞后，线程虽然转入睡眠，却因写组合并从 ~4 op/组 塌缩到 1–2 op/组，吞吐掉到 1/5、CPU/op 反升至 59.9 µs。

有效的结构性优化：打开 enable_pipelined_write（WAL 队列与 memtable 队列分离，写者只需等前序写者的 WAL）后，16 线程写路径 CPU 从 31.1 降到 26.3 µs/op、吞吐

+33%（430→570 MB/s）；32 线程不再塌陷（119→548 MB/s）。在 compaction 受限的 8GB 场景里吞吐不变，但 CPU/op 稳定低 12%——这是唯一在两种瓶颈下都划算的写路径开关。

7 后台线程数的杠杆与代价

图 12　后台线程配置扫描
加 compaction 线程有用但饱和，subcompaction 反而有害
32GB 规模：默认 42 MB/s → 6 线程 66 MB/s（+57%）

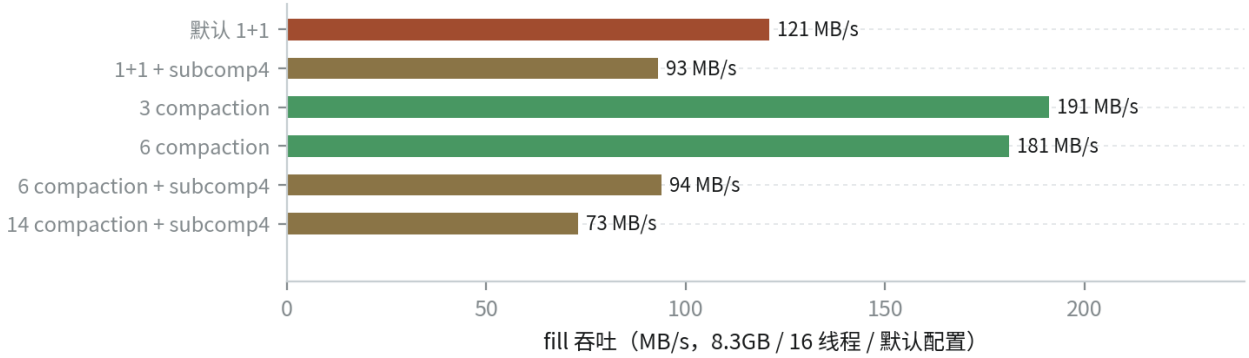


图 12 后台 compaction 线程配置扫描（8.3 GB fill）：1→3/6 线程有正收益，超过 4 个线程饱和；subcompaction 掉 25–34%。

指标	bg2（1+1 线程）	bg8（2+6 线程）	变化
fill 墙钟	4156 s	3148 s	−24%
fill 吞吐	22.9 MB/s	30.3 MB/s	+32%
drain 墙钟	296 s	222 s	−25%
WAF（compaction 读入/应用数据）	9.55×	8.33×	−13%
待压缩字节限速事件	4,402	2,413	−45%
memtable 停车事件	646	858	+33%
未限速区间平均速率	62.1 MB/s	96.9 MB/s	+56%
限速时间占比	74.9%	73.8%	≈
compaction 线程状态	R 59% / D 41%	D 65% / R 24%	转为 IO 等待
flush 单作业 wall（中位）	0.249 s	0.690 s	×2.8
峰值 RSS	1008 MB	1405 MB	+39%

表 7-1 bg2 vs bg8 全指标对照。收益（吞吐、WAF、积压）与代价（flush 延迟、内存）同时增大。

三个要点：① 6 个 compaction 线程把积压清得更快（限速事件 −45%、WAF −13%），但默认 16 MB/s 的限速上限仍在 74% 的时间里生效，所以墙钟只改善 24% 而非数倍；② 磁盘成为新的共享瓶颈——compaction 线程 65% 时间在 D 状态，flush 被挤到单作业延迟 ×2.8，memtable 停车反增；③ 内存 +39%（6 路并发 compaction 的迭代器与输出缓冲 + glibc arena 935→1379 MB），table readers 不变（644 MB，索引块）。

8 内存构成

图 7　内存构成：table readers 是主体，bg8 因 6 路并发 compaction 峰值多 39%

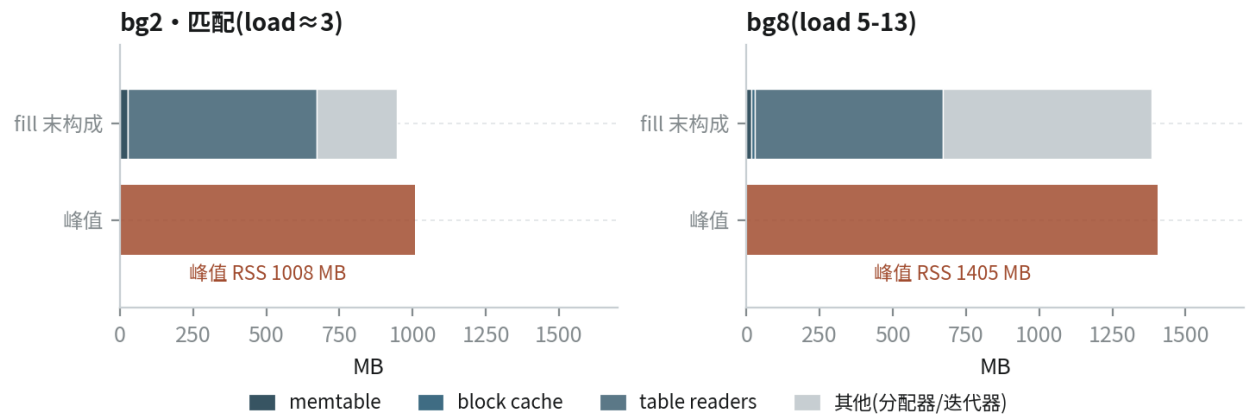


图 7 fill 末的内存构成与峰值 RSS。table readers（SST 索引块）是最大部件，约 6.4 MB/GB 数据。

跑	峰值 RSS	memtable	block cache	table readers	其他	malloc 在用	glibc arena
bg2	1008	27	0.3	643	276	679	935
bg8	1405	15	13.6	644	713	711	1379

表 8-1 内存构成（MB，fill 结束时；峰值 RSS 为全程最大）。

malloc 视角的对账（bg2）：在用堆 679 MB ≈ memtable 27 + 索引块 643 + cache 0.3，说明“其他”并非引擎数据结构，而主要是 glibc 为 memtable arena 保留的空闲堆以及代码/线程栈。block cache 默认仅 32 MB（AutoHyperClockCache），纯写负载几乎不用；真正随数据规模增长的常驻内存是 table readers——打开 SST 的索引块，约 6.4 MB/GB 数据（100 GB ≈ 644 MB，且与后台线程数无关）。

9 线程状态与函数级剖面

图 8　线程状态构成：bg8 的 compaction 线程 65% 时间在等磁盘（D）

R=运行/自旋、S=睡眠/阻塞、D=不可中断 IO 等待；写线程以睡眠为主

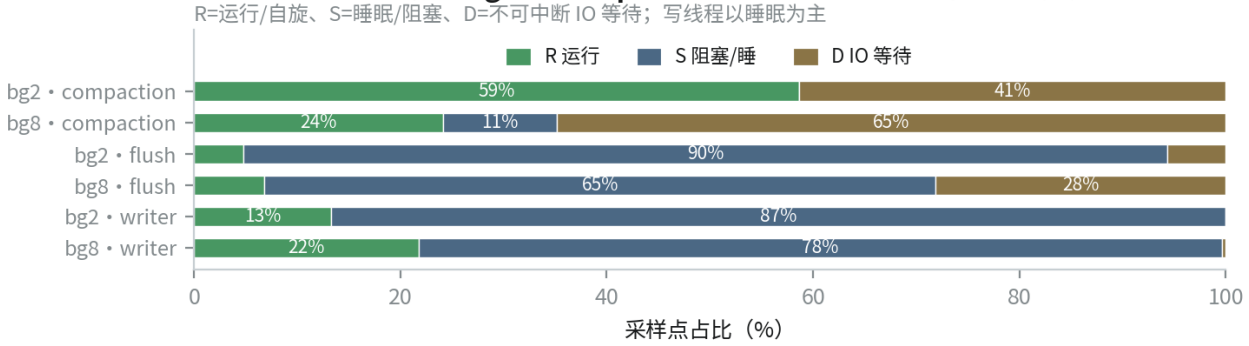


图 8 R/S/D 状态构成。bg8 的 compaction 线程以 IO 等待为主（65% D），写线程以睡眠为主（78–87% S）。

图 10　perf 函数级剖面（bg8，fill 期间 6 轮 × 20 s 采样）
flush / compaction 在搬数据与算校验，writer 在自旋等待

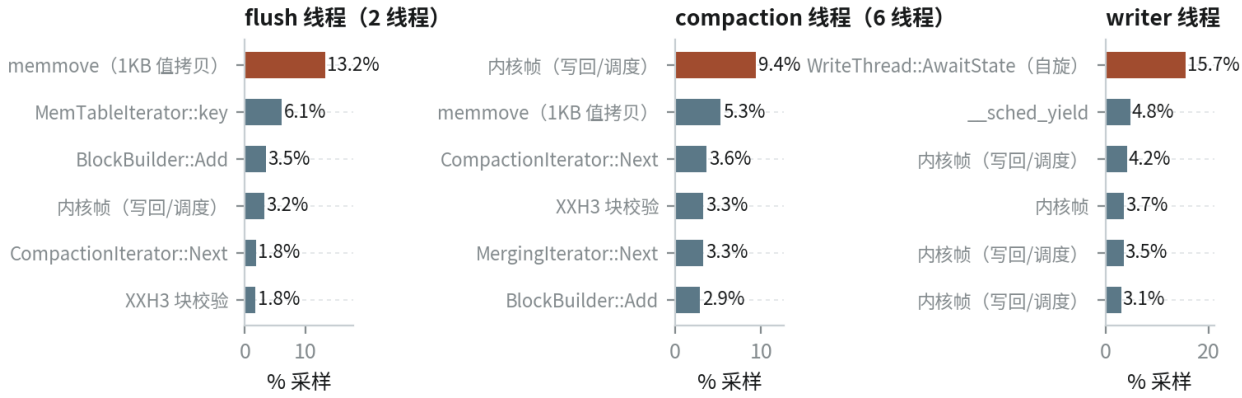


图 10 perf 自时间热点（bg8）：flush/compaction 在搬 1KB 值、迭代 memtable/块与算 XXH3 校验；写线程一多半时间在自旋等待。

把 perf 剖面与 CPU 归因对照，可以得到清晰的成本画像：compaction 的 CPU 花在“读块 → 归并 → 构建新块 → 校验 → 写回”（内核帧 9–14% 是页缓存写回），这部分随数据量线性增长且难以压缩；flush 的成本集中在 memmove（1KB 值拷贝）与块构建；写线程的 CPU 约 20% 直接花在自旋机制上（AwaitState + sched_yield + 内核系统调用帧 + vdso 时钟），而真正插入 memtable 只占 ~2.5%。

这也回答了一个常见误解：“写路径 CPU 74%”并不意味着写入很贵——它衡量的是16 个线程在等待彼此时的机队开销。若要评估引擎的净开销，应看单线程成本（5.76 µs/KV）或同时报告 PerfContext 的 WAL/memtable 分项。

10 结论与优化建议

> 给优化方向的排序（针对 1KB / 100GB / 默认配置）
>
> 1. 解开后台瓶颈优先：限速上限（16 MB/s）与 64 GB 积压软限决定 74% 的墙钟。降低 WAF（更少的跨层重写）或提高 compaction 有效吞吐，是唯一能显著改变吞吐的杠杆；实测 1→6 compaction 线程给 −24% 墙钟、−45% 积压事件。
>
> 2. 写路径结构改造：enable_pipelined_write 是唯一在两种瓶颈下都划算的开关（前台受限 +33% 吞吐；后台受限 −12% CPU）。
>
> 3. 线程数右调：效率最优是单线程；后台受限时 8 线程比 16 线程省 34% CPU、只慢 10%。
>
> 4. 避免反优化：不要开 max_subcompactions；不要靠“减少后台线程数”提速。
>
> 5. 内存侧：table readers 约 6.4 MB/GB 是主要常驻成本；并发 compaction 会额外推高 RSS （6 路时 +39%），显存/内存受限时应限制后台并行度。

10.1 本报告的局限

- 共享机器：外部负载在 3–13 之间波动，bg8 跑时负载更高，故其 24% 的优势为保守下界；绝对值不宜跨负载直接比较，趋势与结构结论不受影响。
- flush 逐作业 CPU 在 bg8（2 个 flush 线程）下不可靠（作业窗口合计 275.5 s > 线程实测 209.5 s）：原因是个别 flush 无 SST 产出、不触发完成回调，导致按 thread_id 的配对错位；驱动已改为按 (thread_id, job_id) 配对。bg8 的 flush CPU 以线程级 209.5 s / cfstats High 行 176.8 s 为准。
- cfstats 的 L0 行在 bg8 中混入了 L0 层 compaction（1900 作业 / 471 s），纯 flush 必须取 High 优先队列行。
- perf 采样窗口覆盖 fill 前中段（每 20 s × 6 轮 × 3 角色），未覆盖 drain；采样需要临时放宽 perf_event_paranoid，跑完已恢复原值。

所有原始数据、脚本与中间产物位于 bench/out/fillcpu_1kb100_bg8/ 与 bench/out/fillcpu_1kb100_bg2matched/，分析脚本见 bench/fillcpu_analyze.py、bench/bg_run_extras.py、bench/fillcpu_compare_bg.py。

附录 A 详细数据表

A.1 三个 100GB 跑的总览

跑	fill s	MB/s	drain s	总 CPU k	WAF	comp 读入 GB	flush 作业	comp 作业	峰值 RSS MB
bg2 · 归档（load 2–3）	3804	25.0	248	11.68	9.52×	951	1550	3410	979
bg2 · 匹配（load ≈3）	4156	22.9	296	10.25	9.55×	953	1552	3376	1008
bg8（load 5–13）	3148	30.3	222	9.90	8.33×	832	1563	3100	1405

A.2 CPU 归因明细（核心 · 秒）

跑	fill 写路径	fill compaction	fill flush	fill 驱动	fill 合计	drain 合计	写路径 µs/KV	写路径占比	comp CPU(库)	flush CPU(库)
bg2 · 匹配	7349	2457	217	80	10103	150	76.5	71.7%	2422	170.9
bg8	6718	2680	210	69	9677	219	70.0	67.9%	2763	176.8

A.3 写线程扫描（wpbench，quiet，2M ops）

线程	ops	wall s	MB/s	CPU µs/op	延迟 µs	utime s	stime s	stime 占比	ops/核 · s
1	2,000,000	13.29	149.2	6.51	6.6	10.8	2.2	17%	153,539
2	2,000,000	11.62	170.7	11.38	11.6	17.4	5.4	24%	87,866
4	2,000,000	7.79	254.7	15.22	15.6	20.7	9.7	32%	65,705
8	2,000,000	5.37	369.2	20.75	21.5	24.2	17.2	42%	48,201
16	2,000,000	4.06	488.1	30.96	32.5	30.9	30.9	50%	32,297
32	2,000,000	15.87	125.0	79.40	254.0	101.8	56.6	36%	12,595

A.4 写路径组件与开关（wpbench）

配置	wall s	写路径 CPU µs/op	MB/s	线程	WAL
仅生成键值（gen，不开库）	4.32	1.08	918	1	1
单线程 Put（WAL 开）	23.97	5.76	166	1	1
单线程 Put（disableWAL）	25.43	4.12	156	1	0
单线程 WriteBatch×8（WAL 开）	18.18	4.43	218	1	1
单线程默认配置（2M ops）	11.28	5.13	176	1	1
16 线程 quiet（WAL 开）	4.32	31.08	459	16	1
16 线程 + 阻塞等待（adaptive_yield=0）	23.15	59.88	86	16	1
16 线程 + disableWAL	6.42	14.14	309	16	0
16 线程默认配置（8GB）	7.19	32.27	276	16	1
16 线程 + pipelined write（非默认）	3.48	26.28	570	16	1
32 线程（默认写路径）	16.62	80.76	119	32	1
32 线程 + pipelined write	3.62	54.79	548	32	1
16 线程默认配置 8GB（pipelined 关）	61.01	40.20	130	16	1
16 线程默认配置 8GB（pipelined 开）	85.51	37.96	93	16	1

表 A-1 wpbench 变体明细（均为 2M ops，除非标注 8GB=8M ops）。pipelined 行的 16/32 线程与 8GB 三行为同批重复测量。

A.5 后台线程 / 子压缩扫描

配置	吞吐 MB/s	写路径 CPU µs/op
bg2_sub1	121	36.8
bg4_sub1	191	33.5
bg8_sub1	181	32.1
bg8_sub4	94	33.0
bg2_sub4	93	35.6
bg16_sub4	73	34.0
bg2_sub1_32GB	42	n/a
bg8_sub1_32GB	66	n/a
bg8_sub4_32GB	48	n/a

A.6 内存构成（MB）

跑	峰值 RSS	memtbl 峰	readers 峰	cache 峰	pending 峰 GB	fill末 memtbl	fill末 readers	fill末 其他	malloc 在用	glibc arena
bg2 · 匹配	1008	128	644	0.3	72	27	643	276	679	935
bg8	1405	128	644	14.1	70	15	644	713	711	1379

A.7 stall 事件与原因

跑	pending compaction bytes delays	memtable limit stops	l0 file count limit delays	l0 file count limit stops	pending compaction bytes stops	total delays	total stops
bg2 · 匹配	4,402	646	158	0	0	4,560	646
bg8	2,413	858	247	0	0	2,660	858

A.8 perf 自时间热点（%）

角色	bg8 热点	bg8 %	bg2 热点	bg2 %
flush	memmove(1KB 拷贝)	13.2	memmove(1KB 拷贝)	14.7
	MemTableIterator::key	6.1	MemTableIterator::key	8.1
	BlockBuilder::Add	3.5	内核帧（写回/调度）	3.4
	内核帧（写回/调度）	3.2	BlockBuilder::Add	3.2
	CompactionIterator::Next	1.8	CompactionIterator::Next	2.6
compaction	内核帧（写回/调度）	9.4	内核帧（写回/调度）	13.5
	memmove(1KB 拷贝)	5.3	memmove(1KB 拷贝)	6.6
	CompactionIterator::Next	3.6	BlockBuilder::Add	3.8
	XXH3 块校验	3.3	MergingIterator::Next	3.8
	MergingIterator::Next	3.3	XXH3 块校验	3.5
writer	WriteThread::AwaitState	15.7	WriteThread::AwaitState	15.5
	__sched_yield	4.8	内核帧（写回/调度）	6.1
	内核帧（写回/调度）	4.2	__sched_yield	3.8
	内核帧（写回/调度）	3.7	内核帧（写回/调度）	3.7
	内核帧（写回/调度）	3.5	内核帧（写回/调度）	2.7

A.9 生效的默认配置（库自报）

选项	默认值
write_buffer_size / max_write_buffer_number	64 MB / 2
max_background_jobs（及其推导）	2 → flush = max(1, 2/4) = 1；compaction = 1；max_subcompactions = 1
level0 触发（trigger / slowdown / stop）	4 / 20 / 36
target_file_size_base / max_bytes_for_level_base	64 MB / 256 MB
level_compaction_dynamic_level_bytes	true
compression	kNoCompression（本机构建未链接压缩库）
block cache（cache_index_and_filter_blocks）	AutoHyperClockCache 32 MB（false）
soft / hard pending compaction bytes	64 GB / 256 GB
delayed_write_rate	16 MB/s（限速上限；按积压 ×0.8 / ×1.25 自适应，下限 16 KB/s）
write_thread_max_yield_usec（adaptive yield）	100 µs（enable_write_thread_adaptive_yield = true）
enable_pipelined_write	false（实测打开可显著改善多写者效率）
max_open_files	-1（table readers 常驻 ≈6.4 MB/GB 数据）